

Análisis y Diseño con Patrones

PEC 1: Principios de Diseño y Patrones de Análisis

En esta segunda PEC evaluaremos principalmente los principios de diseño y los patrones de análisis, contenidos en el temario de la asignatura.

 **NOTA:** Todos los modelos UML que se entreguen deben estar realizados mediante una herramienta de modelado. No se aceptan modelos realizados a mano, ni con herramientas de dibujo tipo *Paint*, ya que estos modelos no tienen rigor. Se recomienda usar la herramienta [VisualParadigm community edition](https://visualparadigm.com/community-edition/).

 **FORMATO:** El documento que entregues con tu propuesta de solución para esta PEC **debe seguir el siguiente formato** (el texto en cursiva corresponde a información que debes rellenar):

Alumno: *...tu nombre...*
Compromiso:
...Certifico que...
Pregunta 1.1:
...tu propuesta de solución...
Pregunta 1.2:
...tu propuesta de solución...
Pregunta 1.3:
...tu propuesta de solución...
...

 **RECUERDA:** Tal como se explica en el Plan docente de la asignatura, esta actividad debe resolverse de forma individual. Por otro lado, no está permitido el uso de herramientas de inteligencia artificial. **En caso de detectar copias o uso de herramientas de IA, se penalizará la actividad con 0 % como nota.**

Para garantizar que has resuelto esta actividad de forma individual y sin uso de herramientas de IA, te pedimos que en tu solución entregada añadas el siguiente compromiso de autorresponsabilidad:

Certifico que he realizado la PEC1 de forma completamente individual y únicamente con la ayuda que el profesorado de esta asignatura considera oportuna, según las indicaciones explicadas en el documento "Originalidad en la evaluación" disponible en el aula. Entiendo que el trabajo no original y/o el empleo de IA generativa implicará que la actividad entregada no será corregida y que automáticamente se le asignará una calificación de 0 %.

Compromiso de autorresponsabilidad

Escribe aquí el texto con el compromiso de autorresponsabilidad que consta en cursiva en la nota previa. **No escribirlo implica la no corrección de la PEC y una calificación de 0 %.**

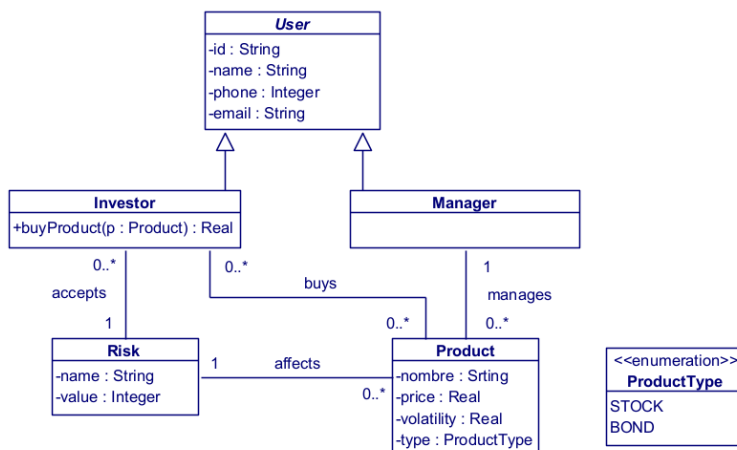
Certifico que he realizado la PEC1 de forma completamente individual y únicamente con la ayuda que el profesorado de esta asignatura considera oportuna, según las indicaciones explicadas en el documento "Originalidad en la evaluación" disponible en el aula. Entiendo que el trabajo no original y/o el empleo de IA generativa implicará que la actividad entregada no será corregida y que automáticamente se le asignará una calificación de 0 %.

Contexto

Todos los ejercicios de esta actividad están relacionados con un hipotético sistema web para realizar inversiones financieras llamado InvestUOC y que permitirá a los usuarios gestionar sus carteras de inversión. Tras las reuniones con los analistas, se han definido los siguientes requisitos iniciales:

Inversores y gestores. El sistema tiene usuarios inversores (*Investors*) y usuarios gestores (*Managers*). De cada uno de ellos nos interesa conocer el identificador, el nombre, el teléfono y su email. Los gestores se encargan de administrar los diferentes productos financieros del sistema y los inversores, o clientes del sistema, son los que adquieren o compran los diferentes productos. Los inversores tienen asignado un nivel de riesgo, en base a un cuestionario que se les hace al darse de alta.

Productos de inversión. Los productos de inversión (*Product*) corresponden con todos los productos financieros que los inversores pueden comprar a través de la plataforma. Por cada producto, el sistema registra el nombre del producto, el precio, la volatilidad y el tipo del producto (acciones o bonos). Adicionalmente, cada producto tiene un nivel de riesgo (*Risk*). El riesgo dispone de un nombre y de un valor numérico. A mayor valor, mayor riesgo.



Restricciones de integridad:

- “User.id” es el identificador del usuario.
- “Risk.name” es el identificador del riesgo.
- “Risk.Value”: Solo puede tomar valores de 1 a 7, que son las categorías de riesgos establecidas a nivel internacional.
- “Product.name” es el identificador de producto.

Partiremos de un sistema/modelo básico al que se le irán añadiendo nuevas funcionalidades a lo largo de los ejercicios que vosotros tenéis que ir resolviendo.

Pregunta 1 (30 %)

En este ejercicio revisaremos los principios de diseño y las violaciones de los mismos en el código.

Una de las funciones más importantes del sistema InvestUOC es la compra de activos financieros por parte de los inversores, y esto es posible gracias al método *buyProduct()* de la clase *Investor*. Al comprar un producto, se debe validar que el riesgo del producto es igual o inferior al riesgo que el inversor está dispuesto a asumir; de este modo se verifica que el inversor no pueda comprar cosas que no encajen con su perfil.

El modelo de negocio de InvestUOC es cobrar una comisión por cada uno de los productos que compran los inversores. Actualmente, la plataforma ofrece acciones (*stocks*) y bonos (*bonds*) que tienen una comisión del 2% y 1%, respectivamente. El método *buyProduct()* devuelve el precio de venta del producto teniendo en cuenta la comisión.

A continuación, se adjunta el pseudocódigo del subsistema descrito en el enunciado.

```
public class Investor extends User {
    private Risk userRisk; // Riesgo de inversión que acepta el inversor
    public void User(...) {}

    /* getters and setters ... */

    public Real buyProduct(Product p) {
        // Validación de riesgo: El inversor comprueba si el producto le encaja
        if (p.getRisk().getValue() > this.userRisk.getValue()) {
            throw new Exception("El producto que está tratando de comprar " +
                                "no es adecuado para el perfil de riesgo del inversor");
        }

        // Cálculo de comisión según el tipo de producto
        Real total = p.getPrice();
        if (p.getType() == ProductType.STOCK) {
            total = total + (total * 0.02); // 2% comisión para acciones
        } else if (p.getType() == ProductType.BOND) {
            total = total + (total * 0.01); // 1% comisión para bonos
        }
        this.registerPurchase(p);
        return total;
    }
}
```

Pregunta 1.1 (5 %). Enumera y explica los principios de diseño que no se cumplen en la descripción y pseudocódigo proporcionados en el ejercicio 1.

Pregunta 1.2 (15 %). Proporciona un modelo de clases UML que corrija los principios que no se cumplen según el apartado anterior; y explica brevemente el porqué de los cambios realizados. No hace falta explicar detalladamente cada elemento del diagrama, solamente los cambios principales.

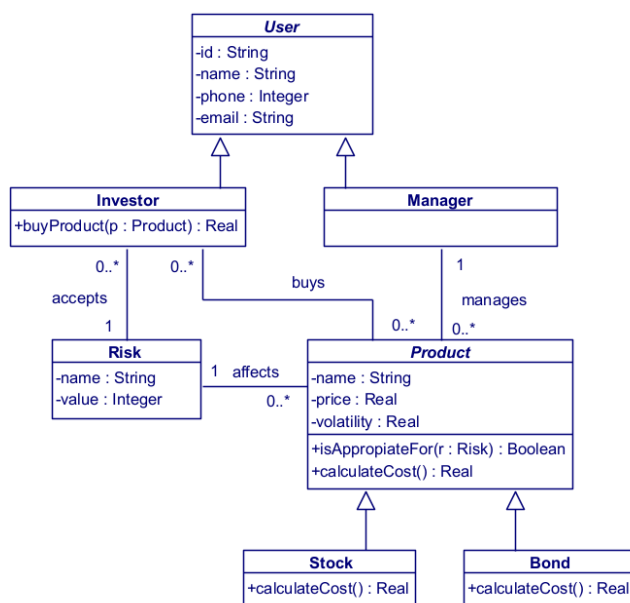
Pregunta 1.3 (10 %). Proporciona el pseudocódigo que corrige los principios que no se cumplen según la pregunta 1.1 y que sea coherente con el nuevo modelo de la pregunta 1.2.

Solución

Pregunta 1.1. Los principios que se incumplen y el porqué serían los siguientes:

- Ley de Demeter (no hables con extraños). Ya que para validar si el riesgo del inversor es adecuado, accede al riesgo del producto navegando a través de este `p.getRisk().getValue()`. Según este principio, no se debería acceder a los componentes internos de objetos.
- Principio de Abierto Cerrado (OCP). Se incluye un bloque `if-else` para calcular el precio final. Si se modifica la comisión de los diferentes productos o se añade uno nuevo, el código se debería modificar.
- Alta cohesión. Este principio se incumple, ya que la clase “Investor” asume la responsabilidad de calcular las comisiones y validar el riesgo, tareas que a priori no le corresponden.

Pregunta 1.2. En este caso, para resolver los problemas relacionados con los principios anteriormente incumplidos, se ha convertido la clase *Product* en una clase abstracta y se han creado las clases *Stock* y *Bond* que heredan de la clase *Product*. De este modo, cada tipo de producto se encargará de realizar el cálculo de su precio. Adicionalmente, a la clase *Product* se le añade un método para ver si el producto es adecuado para un riesgo determinado.



Restricciones de integridad

- “User.id” es el identificador del usuario.
- “Risk.name” es el identificador del riesgo.
- “Risk.Value”: Solo puede tomar valores de 1 a 7 que son las categorías de riesgos establecidas a nivel internacional.
- “Product.name” es el identificador de producto

Pregunta 1.3. A continuación se muestra el pseudocódigo.

```
public class Investor extends User {
    private Risk userRisk;

    public Real buyProduct(Product p) {
        /* Solución Ley Demeter y alta cohesión: Se delega la validación del riesgo
         * al producto, pasándole como parámetro el riesgo del usuario*/
        if (!p.isAppropriateFor(this.userRisk)) {
            throw new Exception("El producto que está tratando de comprar no es " +
                                "adecuado para el perfil de riesgo del inversor");
        }
        /* Solución OCP y alta cohesión: En vez de calcular el inversor el coste, se
         * delega este cálculo a cada producto*/
        Real total = p.calculateCost();
        this.registerPurchase(p);
        return total;
    }
}

Public abstract class Product {
    protected Real price;
    protected Risk risk;

    public abstract Real calculateCost();

    public boolean isAppropriateFor(Risk r) {
        /* Se valida que el riesgo que se le pasa es mayor o igual que el riesgo
         * del producto */
        return this.risk.getValue() <= r.getValue();
    }
}

public class Stock extends Product {
    public Real calculateTotalCost() {
        return this.price * 1.02; // 2% comisión
    }
}

public class Bond extends Product {
    public Real calculateTotalCost() {
        return this.price * 1.01; // 1% comisión
    }
}
```

Pregunta 2 (20 %)

Tras el éxito inicial, la plataforma InvestUOC ha decidido internacionalizar su catálogo, permitiendo la comercialización de activos financieros procedentes de diversas regiones geográficas. Los productos pueden cotizar en monedas diferentes al euro (EU), específicamente en dólares (USD) y libras esterlinas (GBP). El sistema debe ser capaz de gestionar los precios teniendo en cuenta la moneda.

La gerencia ha observado que algunas operaciones generan beneficios que ni siquiera cubren el coste de la operación (márgenes "irrisorios"), mientras que otras imponen recargos que los inversores consideran "abusivos". Se necesita la implementación de una estructura que defina una horquilla de rentabilidad para las comisiones que se aplican a cada producto. Las rentabilidades deben expresarse en valor absoluto y no en porcentaje, y deben tener en cuenta las diferentes regiones geográficas. Es vital que el diseño permita cambiar la horquilla de las comisiones sin que esto afecte a la estructura de la clase *Product*, asegurando que la lógica de validación de magnitudes no quede dispersa por el sistema.

Pregunta 2.1 (5 %). Identificar qué patrones de análisis deben aplicarse para diseñar el sistema, justificando brevemente su utilidad.

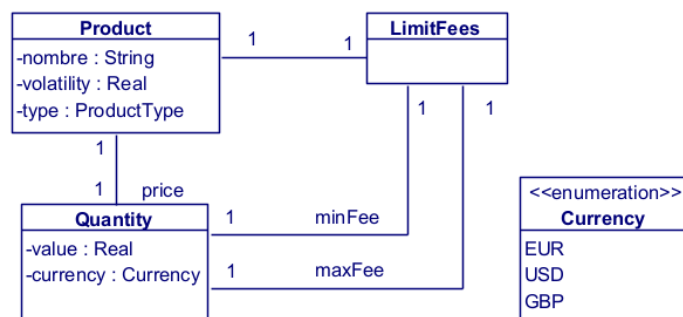
Pregunta 2.2 (15 %). Proponer un diagrama estático de análisis, aplicando los patrones identificados. Usa de base el diagrama del enunciado; puedes dibujar solo la parte del modelo necesaria para explicar el ejercicio. No hace falta explicar detalladamente cada elemento del diagrama, solamente los cambios principales.

Solución

Pregunta 2.1. En este caso sería conveniente aplicar 2 patrones:

- El patrón **Cantidad** nos permite gestionar los precios en varias monedas. Este patrón se aplicará a los precios de los productos y también se deberá aplicar para las comisiones. De esta manera, el sistema permite que para cada producto se pueda especificar un valor en la moneda en la que opere dicho producto. Adicionalmente, deja el sistema abierto a que se puedan aplicar conversiones de monedas en un futuro.
- A la hora de definir la horquilla o los límites que se aceptan para las comisiones, el patrón **Rango** sería el adecuado, ya que permite definir los límites a aplicar a las comisiones que se pueden aplicar a la compra y venta de los productos. Adicionalmente, el patrón cantidad se aplicaría de manera combinada con el rango, ya que los valores de las comisiones también se ven afectados por la moneda.

Pregunta 2.2.



Pregunta 3 (30 %)

Debido a la creciente demanda de diversificación, InvestUOC ha decidido evolucionar su catálogo de servicios mediante la introducción de Instrumentos Financieros Estructurados. A diferencia de los activos básicos (como acciones o bonos), estos instrumentos operan como vehículos de agregación que permiten combinar diferentes activos bajo una única denominación comercial.

Una particularidad crítica de estos nuevos instrumentos es su naturaleza recursiva: un instrumento estructurado puede estar conformado por un conjunto de activos individuales, pero el sistema también debe permitir que una estrategia de inversión incorpore otras estrategias ya existentes de menor nivel, formando así una jerarquía de activos anidados.

Adicionalmente a la introducción de los instrumentos financieros, el sistema debe incluir un mecanismo que permita calcular el valor del portfolio del cliente, entendiendo como portfolio la lista de productos que un cliente ha comprado.

Pregunta 3.1 (5 %). ¿Qué patrón o patrones de análisis de los vistos en la asignatura serían interesantes utilizar para poder implementar esta funcionalidad?

Pregunta 3.2 (15 %). Proporcionar el modelo de clases UML que implemente la funcionalidad descrita en el apartado anterior. No olvides añadir las restricciones de integridad. Utiliza el modelo inicial del enunciado como base para hacer el nuevo modelo. No hace falta explicar detalladamente cada elemento del diagrama, solamente los cambios principales.

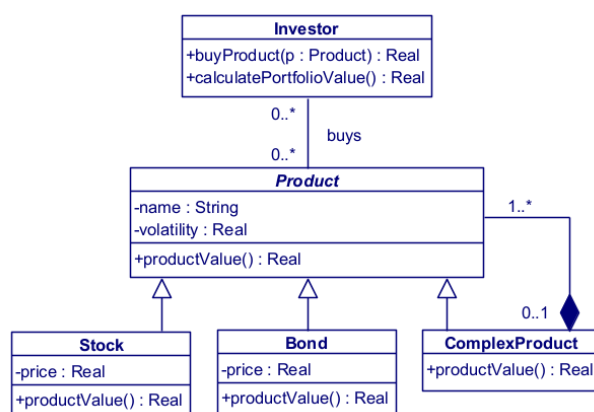
Pregunta 3.3 (10 %). Proporciona el pseudocódigo de la funcionalidad que permite calcular el valor del portfolio del cliente, cumpliendo los principios. El pseudocódigo debe incluir todo el código relacionado con esta funcionalidad,

 **NOTA:** En tus respuestas deberás considerar solo las clases que se mencionan en el enunciado de la pregunta 3

Solución

Pregunta 3.1. En este caso, el patrón a utilizar sería el **Objeto Compuesto**, que nos permite tratar todos los productos de manera homogénea. Adicionalmente, permite que el objeto compuesto pueda contener otros objetos de manera recursiva. En este caso, los bonos y las acciones serían objetos simples; los instrumentos financieros serían objetos compuestos que pueden tener una lista de productos, independientemente de que sean simples o compuestos.

Pregunta 3.2. El modelo de clases sería el siguiente:



Pregunta 3.3.

```

public class Investor {
    private ListProducts portfolio;

    /* Constructor */
    /* getters y setters */
    public Real calculatePortfolioValue() {
        Real total = 0.0;
        for (Product item : portfolio) {
            total = total + item.productValue();
        }
        return total;
    }
}

public abstract class Product {
    protected String name;
    protected Real volatility;
    /* Constructor */
    /* getters y setters */
    public abstract Real productValue();
}

public class Stock extends Product {
    private Real price;
  
```

```
    /* Constructor */
    /* getters y setters */
    public Real productValue() {
        return this.price; // o cualquier fórmula que se aplique
    }
}

public class Bond extends Product {
    private Real price;
    /* Constructor */
    /* getters y setters */
    public Real productValue() {
        return this.price; // o cualquier fórmula que se aplique
    }
}

public class ComplexProduct extends Product {
    private ListProduct items;
    /* Constructor */
    /* getters y setters */
    public Real productValue() {
        Real sum = 0.0;
        for (Product item : items) {
            sum += item.productValue();
        }
        return sum;
    }
}
```

Pregunta 4 (20 %)

Partiendo del enunciado inicial. Tras la inclusión inicial de los productos de acciones (*Stock*) bonos (*Bond*), la plataforma InvestUOC decide añadir dos nuevos productos financieros para diversificar la oferta con dos tipos de productos nuevos:

- *CryptoAsset*: Hace referencia a activos digitales que operan sobre una red *blockchain*, el más típico de este tipo de activos es Bitcoin, Ethereum y Solana.
- *Deposit* (Depósito a plazo fijo): Es un producto bancario que se consume mucho en España ya que no tiene riesgo y tiene una fecha de vencimiento fija en la que se liquida el activo y se recupera la cantidad aportada más los beneficios generados.

Al haber ya 4 tipos de productos y con peculiaridades especiales, el equipo de desarrollo ha decidido añadir a la clase abstracta *Product* y crear una clase que hereda de *Product* para cada tipo de producto.

A continuación se muestra el pseudocódigo de cómo quedarían la clase *Product* y las clases para los diferentes tipos de producto:

```
public abstract class Product {
    protected Real price;
    public abstract Real getPrice();
    public abstract String getBlockchainAddress();
    public abstract Date getMaturityDate();
}

public class Stock extends Product {
    public Real getPrice() { return this.price * 1.02; }
    public String getBlockchainAddress() { return null; }
    public Date getMaturityDate() { return null; }
}

public class Bond extends Product {
    public Real getPrice() { return this.price * 1.01; }
    public String getBlockchainAddress() { return null; }
    public Date getMaturityDate() { return null; }
}

public class CryptoAsset extends Product {
    public String address;
    public Real getPrice() { return this.price; } // no tiene comision
    public String getBlockchainAddress() { return address; }
    public Date getMaturityDate() { return null; }
}

public class Deposit extends Product {
    private boolean isLocked; // En determinados momentos del día no se puede consultar
                             // el valor del fondo y queda marcado como bloqueado

    public Real getPrice(){
        if (this.isLocked) {
            return this.price
        }
        return this.price + this.price*1/365;
    }

    public String getBlockchainAddress() { return null; }
    public Date getMaturityDate() { return new Date("2029-01-01"); }
```

}

Pregunta 4.1 (2.5 %). Indica si este diseño satisface o no el principio de segregación de interfaces. Razona tu respuesta.

Pregunta 4.2 (2.5 %). Indica si este diseño satisface o no el principio de sustitución de Liskov. Razona tu respuesta.

Pregunta 4.3 (15 %). Proporciona pseudocódigo que arregle los dos principios vistos en las preguntas 4.1 y 4.2, si fuera necesario.

Solución

Pregunta 4.1. En este caso se incumple el principio de segregación de interfaces, ya que la clase producto dispone de dos métodos *getBlockchainAddress()* y *getMaturityDate()*, que obliga a reescribir a todas las clases hijas. Según el principio, las clases no deberían tener métodos que no usen y se deberían resolver mediante interfaces.

Pregunta 4.2. En este caso no se incumple el principio de sustitución de Liskov

Pregunta 4.3

```
public abstract class Product {
    protected Real price;
    public abstract Real getPrice();
}

public interface IBlockchain {
    public String getBlockchainAddress();
}

public interface IMaturity {
    public Date getMaturityDate();
}

public class Stock extends Product {
    public Real getPrice() { return this.price * 1.02; }
}

public class Bond extends Product {
    public Real getPrice() { return this.price * 1.02; }
}

public class CryptoAsset extends Product implements IBlockchain {
    public String address;
    public Real getPrice() { return this.price; } // no tiene comision
    public String getBlockchainAddress() { return address; }
}

public class Deposit extends Product implements IMaturity {
    private boolean isLocked; // En determinados momentos del día no se puede consultar
                             // el valor del fondo y queda marcado como bloqueado
    public Real getPrice(){
        if (this.isLocked) {
            this.price
        }
        return this.price + this.price*1/365;
    }
    public Date getMaturityDate() { return new Date("2029-01-01"); }
}
```

Criterios de evaluación

La nota final de la PEC se calculará a partir de las notas obtenidas en cada una de las preguntas individuales, con el peso indicado en el enunciado. En concreto, este es el criterio:

- Pregunta 1: 30 %
- Pregunta 2: 20 %
- Pregunta 3: 30 %
- Pregunta 4: 20 %

Esta PEC corresponde a un **45 % de la nota de Evaluación Continua de la asignatura**.



RECUERDA: Para acogerte a la evaluación continua de la asignatura, deberás entregar todas las PECs y PRAs y **superar cada una de ellas con al menos 3 puntos**.

Tal como se explica en el plan docente de la asignatura, esta actividad debe resolverse de forma individual. Por otro lado, no está permitido el uso de herramientas de inteligencia artificial. Recuerda que debes incluir el texto indicado en la sección "Compromiso de autorresponsabilidad" de tu PEC. En caso de detectar copias o uso de herramientas de IA, se penalizará la actividad con un 0 % como nota.

Formato de entrega

- Deberás entregar un **único documento** (formato pdf o docx) con tu propuesta de solución para las preguntas del enunciado.
- Recuerda que debes producir diagramas **conforme a la especificación UML** y **ser consistente** en el estilo de diagramas que entregues en todos los ejercicios.
- Todos los documentos se tienen que entregar en el aula virtual antes de las 23:59 horas del **24 de marzo de 2026**.
- **No se aceptarán entregas fuera de plazo.**

Propiedad intelectual y plagio

La Normativa académica de la UOC dispone que el proceso de evaluación se cimenta en el trabajo personal del estudiante y presupone la autenticidad de la autoría y la originalidad de los ejercicios realizados.

La ausencia de originalidad en la autoría o el mal uso de las condiciones en las que se realiza la evaluación de la asignatura, es una infracción que puede tener consecuencias académicas graves.

Te recomendamos leer esta [guía sobre el plagio académico y como evitarlo](#). Recuerda que siempre que reutilices contenido de terceros, debes citar la fuente. Puedes leer también esta [guía para saber cómo citar correctamente](#).

El estudiante será calificado con un suspenso (0 %) si se detecta falta de originalidad en la autoría, sea porque haya utilizado material o dispositivos no autorizados, sea porque ha copiado textualmente de internet, o ha copiado apuntes, de otras PECs, de materiales, manuales o artículos (sin la cita correspondiente) o de otro estudiante, o por cualquier otra conducta irregular.